



بسمه

تعالی



ش پایانی مسئله نهایی «محور اصلی مسابقه کی بُرد هوش مصنوعی»

شاخه تخصصی پردازش تصویر

شماره گروه	نام و نام خانوادگی عضو یا اعضا	پایه
39	رادین الماسی-علی نجف پور	11

استخراج خودکار اطلاعات شخصی از کارت‌های ملی با استفاده از پردازش تصویر و OCR

خلاصه:

در این پروژه، ما یک سیستم هوش مصنوعی را با هدف استخراج خودکار اطلاعات از کارت‌های ملی طراحی و توسعه دادیم. هدف اصلی این پروژه، شناسایی و استخراج اطلاعات کلیدی شامل نام، نام خانوادگی، کد ملی، نام پدر، تاریخ تولد و تاریخ انقضای کارت بود. برای دستیابی به این هدف، رویکردی سه‌مرحله‌ای را پیاده‌سازی کردیم. در مرحله نخست، با بهره‌گیری از مدل YOLO (You Only Look Once)، کارت ملی را از سایر بخش‌های تصویر ورودی جدا نمودیم. سپس با انجام چند عملیات دیگر چرخش آن و پرسپکتیو آن را درست کردیم. سپس، با استفاده از مدل دیگری از YOLO، نواحی مربوط به هر یک از فیلدهای اطلاعاتی را شناسایی و برش دادیم. در نهایت، با به‌کارگیری مدل تنظیم‌شده‌ی "هزار" که برای تشخیص نوری کاراکترها (OCR) بهینه‌سازی شده بود، متن موجود در این نواحی را استخراج کردیم و نتایج را در قالب فایل CSV ارائه دادیم. ارزیابی‌های انجام‌شده نشان داد که این سیستم در اکثر مراحل به دقتی بیش از 90 درصد دست یافته است. با این حال، در پردازش تصاویری با کیفیت بسیار پایین، همچنان چالش‌هایی وجود دارد که نیاز به بهبود در آینده را نشان می‌دهد.

مقدمه:

کارت ملی به عنوان یکی از مهمترین مدارک هویتی در هر کشور، اطلاعات کلیدی هویت افراد را در بر دارد و در سیستم های مختلف بانکی، قضایی و اداری کاربردهای گسترده ای دارد. با گسترش روزافزون خدمات بانکداری الکترونیک، دولت هوشمند و پلتفرم های آنلاین مالی مانند صرافی ها، نیاز به مدارک هویتی ساده اما با امنیت بالا بیش از پیش احساس می شود. در این میان، احراز هویت دیجیتال افراد برای دولت ها و نهادهای مالی از اهمیت بسزایی برخوردار است، زیرا این فرآیند به پیشگیری از جرایمی مانند پولشویی و کلاهبرداری کمک می کند. به همین منظور، بسیاری از بانک ها، صرافی ها و سایر نهادها از سیستم های احراز هویت آنلاین استفاده می کنند که یکی از مراحل اصلی آن، بارگذاری مدارک هویتی است.

با این حال، استخراج اطلاعات از این مدارک به صورت دستی و با روش های سنتی، فرآیندی زمان بر، پرهزینه و مستعد خطا است. این پروژه با هدف رفع این چالش طراحی شده و با بهره گیری از تکنیک های نوین پردازش تصویر و هوش مصنوعی، به ویژه فناوری تشخیص نوری کاراکترها (OCR)، امکان استخراج خودکار، سریع و دقیق اطلاعات هویتی از کارت های ملی را فراهم می کند. این رویکرد نه تنها سرعت انجام فرآیند را به طور قابل توجهی افزایش می دهد، بلکه هزینه های مرتبط با احراز هویت را نیز به شکل چشمگیری کاهش می دهد.

روش شناسی:

در این پروژه، ما از همان ابتدای کار، مراحل را به چهار بخش اصلی تقسیم کردیم تا بتوانیم به صورت نظاممند کار خود را انجام دهیم این چهار بخش عبارت بودند از:

1. **تشخیص کارت ملی:** در این مرحله، هدف ما جداسازی کارت ملی از تصویر ورودی بود. برای این منظور، از مدل YOLO استفاده کردیم تا کارت ملی را شناسایی کرده و آن را به صورت کامل یا حداقل در قالب یک مستطیل که کارت را در بر بگیرد، از سایر بخش‌های تصویر جدا کنیم.
2. **انجام عملیات چرخش و تنظیم پرسپکتیو:** مشکلی که بیش از همه در مسیر پروژه با آن روبه‌رو بودیم
3. **تشخیص نواحی اطلاعاتی:** پس از جداسازی کارت، نواحی حاوی اطلاعات کلیدی مانند نام، نام خانوادگی، کد ملی و غیره را با استفاده از یک مدل YOLO دیگر شناسایی و برش دادیم تا برای مرحله بعدی آماده شوند.
4. **استخراج متن:** در نهایت، با بهره‌گیری از یک مدل OCR که برای زبان فارسی بهینه شده بود، متن موجود در نواحی شناسایی‌شده را استخراج کردیم.

چالش‌های اصلی پروژه

در طول این پروژه، با چند مشکل اساسی روبه‌رو شدیم:

- نبود دیتاست آماده: هیچ مجموعه داده آماده‌ای برای آموزش مدل‌ها در توسط کی برد در اختیار ما قرار نگرفت بود پس ما باید خودمون اینکار را انجام میدادیم.
- چرخش و پرسپکتیو تصویر
- کیفیت پایین تصاویر
- محدودیت‌های مدل OCR برای زبان فارسی

راه حل‌ها و اقدامات انجام‌شده

جمع‌آوری و آماده‌سازی دیتاست

- ابتدا از اطرافیان خود تعدادی کارت ملی جمع‌آوری کردیم و از آن‌ها در زوایا و مدل‌های مختلف عکس‌برداری نمودیم. این کار حدود 50 تصویر در اختیار ما قرار داد، اما به وضوح کافی نبود و برچسب‌گذاری آن‌ها نیز زمان‌بر و دشوار بود.
- به همین دلیل، به جست‌وجو در اینترنت پرداختیم و با سایت Roboflow آشنا شدیم. این پلتفرم ابزاری تخصصی برای پروژه‌های بینایی ماشین و پردازش تصویر است که به‌ویژه در جمع‌آوری

- و برجسب‌گذاری تصاویر کاربرد دارد. در بخش Universe این سایت، با پروژه‌های متعددی مواجه شدیم که پیش‌تر با کارت ملی کار کرده بودند. ما تصاویر خام را به پروژه خود اضافه کردیم و با استفاده از ابزارهای Roboflow، آن‌ها را برجسب‌گذاری کردیم.
- در نهایت، توانستیم برای هر یک از مراحل پروژه بین 1500 تا 2000 داده برای آموزش مدل‌ها فراهم کنیم.
- برای دیتاست مربوط به تقویت مدل OCR، یک اسکریپت نوشتیم که به صورت خودکار کارت ملی و متن آن را تولید کند. با این روش، حدود 1500 عکس برای تنظیم دقیق (Fine-tuning) مدل OCR آماده کردیم.

آموزش مدل برای جداسازی کارت ملی از پس زمینه

در مرحله اول، با استفاده از دیتاست تهیه‌شده، مدل YOLO را برای تشخیص کارت ملی آموزش دادیم. این مدل پس از آموزش به دقتی در حدود 91% رسید (تصویر 1).

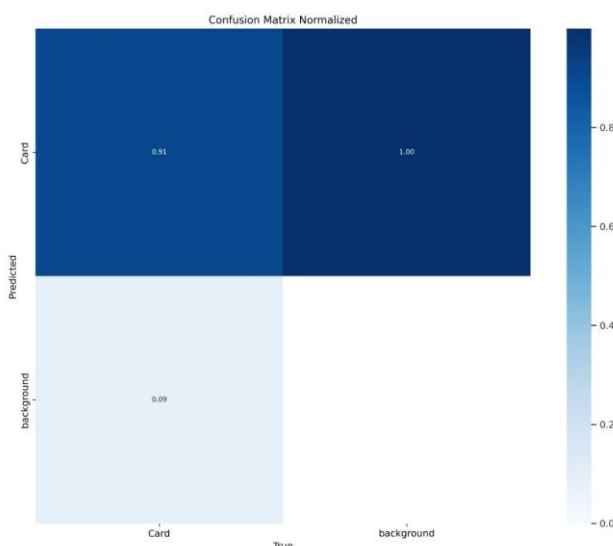


Figure 1 ماتریس در هم ریختگی مدل Card Detection

این اولین روش انجام کار بود هدف این روش این بود که کارت ملی را در تصویر ورودی شناسایی کنیم و آن را به صورت یک ناحیه مستطیل (Bounding Box) جدا کنیم

دومین روشی که برای این کار امتحان کردیم نوشتن یک مدل YOLO-Segmentation بود که خیلی دقیق‌تر مرزها را مشخص بکند تا بتوانیم خیلی راحت‌تر برای چرخش پرسپکتیو و استخراج متن از آن استفاده بکنیم

برای این مدل هم باز هم مثل قبل با کمک roboflow جمع‌آوری تصاویر و برجسب‌گذاری را انجام دادیم و در نهایت بعد از 2100augmentation تا عکس برای آموزش در اختیار داشتیم و از این مدل هم دقتی حدود 95% را خروجی گرفتیم (تصویر 2)

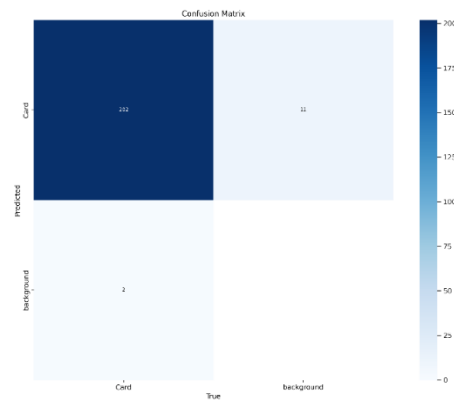


Figure 2 ماتریس در هم ریختگی مدل card segmentation

مقایسه خروجی دو روش در یک نگاه



Figure 4 مدل card detection



Figure 3 مدل card segmentation

به وضوح قابل مشاهده است که روش دوم یعنی segmentation بسیار کاربردی تر و دقیق تر است و نیاز ما در این پروژه را برآورده میکند. حال با نوشتن چند خط کد دیگر و استخراج کارت با دقت بسیار بالاتری میتوانیم با سرعت بیشتر و دقت خیلی بیشتر فرایند چرخش و پرسپکتیو را روی کارت اعمال کنیم

```
if len(approx) == 4:
    # reshape and sort corners: top-left, top-right, bottom-right, bottom-left
    corners = approx.reshape(4, 2)
    corners = sorted(corners, key=lambda x: x[1]) # sort by y-coordinate
    top_corners = sorted(corners[:2], key=lambda x: x[0]) # sort top by x
    bottom_corners = sorted(corners[2:], key=lambda x: x[0]) # sort bottom by x
    ordered_corners = [top_corners[0], top_corners[1], bottom_corners[1], bottom_corners[0]]

    # calculate bounding rectangle and aspect ratio
    x, y, w, h = cv2.boundingRect(contour)
```

انجام چرخش و پرسپکتیو روی تصاویر

همان طور که اشاره شد ما در نهایت برای مدل OCR از fine-tune کردن مدل هزار کمک گرفتیم مدلی که مخصوصا برای زبان فارسی توسعه داده شده و دقت بسیار بالایی دارد.

اگر البته که تصاویر ورودی کاملاً تراز بوده و فاقد هرگونه چرخش یا پرسپکتیو باشند به همین دلیل مرحله انجام چرخش و پرسپکتیو به نوعی مهمترین بخش پروژه ما بود در راستا این قضیه اولین روشی که پیش گرفته شد روش کلاسیک گوشه یابی بود . در این روش، هدف شناسایی گوشه‌های کارت ملی و استفاده از آن‌ها برای محاسبه زاویه چرخش و تبدیل پرسپکتیو مورد نیاز بود.

عواملی نظیر کیفیت پایین برخی تصاویر، وجود نویز، زوایای متنوع و پس‌زمینه‌های پیچیده، دقت تشخیص گوشه‌ها را به شدت کاهش داد. پس از انجام چند آزمایش اولیه، مشخص شد که این روش از کارایی لازم برخوردار نیست و به همین دلیل کنار گذاشته شد.

دومین راه حل ما استفاده از روش مبتنی بر تشخیص چهره بود . در ابتدا ما با استفاده از مدل MTCNN چهره داخل هر کارت ملی را تشخیص داده سپس با محاسبه زاویه بین مرکز چهره و مرکز کارت زاویه مناسب برای چرخش را مشخص کرده و سپس با استفاده از کتابخانه های CV2 چرخش را اعمال کردیم

```
detector = MTCNN()

def rotation(path):
    image = cv2.imread(path) # set image
    results = card_model(image) # run card yolo model for image
    if len(results[0].boxes) == 0: # if cant recognize card
        print("No card detected!")
        return None

    # set 4 corner location
    x1, y1, x2, y2 = map(int, results[0].boxes[0].xyxy[0])
    cropped_card = image[y1:y2, x1:x2]

    # check for faces
    faces = detector.detect_faces(cropped_card)
    if faces:
        x, y, w, h = faces[0]['box']
        face_center = (x + w // 2, y + h // 2) # face center location
        card_center = (cropped_card.shape[1] // 2, cropped_card.shape[0] // 2) # card
        center location
        angle = np.arctan2(face_center[1] - card_center[1], face_center[0] -
        card_center[0]) * (180 / np.pi) + 180 # angle of rotation
        rotation_matrix = cv2.getRotationMatrix2D(card_center, angle, 1.0) # make matrix
        for rotation
        rotated = cv2.warpAffine(cropped_card, rotation_matrix, (cropped_card.shape[1],
        cropped_card.shape[0])) # rotation image
        return rotated # return final image result
    else:
        # if cant find face
        return smart_rotation(cropped_card)[0]
```

این مدل به خوبی نیاز ما برای فاز 2 مسابقه را فراهم کرد و در حدود 70 درصد مواقع چهره را به خوبی تشخیص داده و چرخش را انجام میداد ولی همچنان مشکلاتی وجود داشت . در وهله اول سرعت پایین مدل MTCNN بود و سپس مواقعی که قادر به تشخیص چهره نبود و ما نیاز داشتیم که همچنان دقت مدل را افزایش دهیم.

به همین دلیل با انجام جست و جو های بیشتر و امتحان کردن چندین مدل مختلف به مدل buffalo_I برخوردیم که سرعت و دقت را همزمان برای پروژه ما همراه داشت.

مقایسه نتایج حاصل از اعمال MTCNN و buffalo_I:

	image_id	national_id	first_name	last_name	birth_year	birth_month	birth_day	father_name	expiry_year	expiry_month	expiry_day
0	IMG_20250313_224003_750	۴۲۷۱۵۵۴۰۵۷	کیران	الهاسی	۱۳۸۱	۰۶	۱۱	بیگ	۱۴۰۳	۰۷	۲۴
5	IMG_20250313_224004_014	۹۴۹۰۵۴۹۰۰۸	اکبر	کرمی	۱۳۶۷	۱۱	۲۳	نیرداد	۱۴۱۰	۰۳	۲۰
3	IMG_20250313_224004_033	۸۲۳۳۹۷۳۸۲۳	های	علینی	NaN	NaN	NaN	آر	۲۱	آ	۴۰
4	IMG_20250313_224004_369	۵۹۵۹۰۶۶۳۸۹	مصطفی	بختیاری	۱۳۸۲	۰۶	۰۹	عصمتی	۱۴۰۷	۰۷	۱۷
2	IMG_20250313_224004_534	۲۴۷۰۹۴۹۷۱۱	نرانه	هزاری	۱۳۷۰	۱۰	۱۳	انریک	۱۴۰۵	۰۶	۲۶

Figure 6 مدل MTCNN

	image_id	national_id	first_name	last_name	birth_year	birth_month	birth_day	father_name	expiry_year	expiry_month	expiry_day
4	20	۴۲۷۱۵۵۴۰۵۷	کیران	الهاسی	۱۳۸۱	۰۶	۱۱	مجد	۱۴۰۳	۰۷	۲۴
0	46	۸۲۳۳۹۷۳۸۲۳	های	علینی	۱۳۸۱	۰۸	۲۹	آرش	۱۴۰۵	۱۰	۲۱
2	98	۳۴۳۱۶۹۸۳۵۸	ساحل	روایتش	۱۳۶۵	۰۵	۲۱	بهمن	۱۴۰۹	۱۰	۲۱
1	105	۹۴۹۰۵۴۹۰۰۸	اکبر	کرمی	۱۳۶۷	۱۱	۲۳	نیرداد	۱۴۱۰	۰۳	۲۰
5	1591	۵۹۵۹۰۶۶۳۸۹	مصطفی	بختیاری	۱۳۸۲	۰۶	۰۹	عصمتی	۱۴۰۷	۰۷	۱۷
3	2145	۲۴۷۰۹۴۹۷۱۱	نرانه	هزاری	۱۳۷۰	۱۰	۱۳	انریک	۱۴۰۵	۰۶	۲۶

Figure 5 مدل buffalo_I

این روش به نظر خوب میرسید ولی بعد از صحبت با منتور ها تصمیم گرفتیم که یک روش دیگر را هم امتحان کنیم و نتایج حاصل از آن را ببینیم به همین دلیل به دنبال روش سوم رفتیم سومین روش دوباره برپایه یک مدل YOLO بود در این روش ما دوباره با استفاده از یک مدل YOLO دو شکل مهم را که در همه کارت ملی ها مشترک بودند را پیدا میکردیم یعنی پرچم ایران و کلمه و علامت الله در کارت ملی در اینجا ما با شناسایی این دو شی در شکل و محاسبه زاویه خط بین مرکز این دو شکل داشت. این سیستم دقت مورد قبولی را در مقابل مدل MTCNN ارائه میداد ولی در مقایسه با مدل buffalo_I عملکرد ضعیفتری داشت هم از نظر سرعت و هم از نظر دقت و البته این مشکل که در بیشتر مواقع خروجی آن نیاز به چرخش 180 دیگر داشت که این به این معنی بود که یک مرحله دیگر باید به کد ما اضافه میشد که هم از نظر سرعت اجرا و تمیزی کد خروجی به صرفه به نظر نمیرسید به همین دلیل در نهایت تصمیم گرفته شد که از سیستم تشخیص چهره استفاده شود

کد و نحوه عملکرد سیستم سوم (تشخیص با دو لوگو موجود در کارت ملی)

```

def rotate_idcard(image_path):
    image = cv2.imread(image_path) # set image
    results = card_model(image) # run card yolo model for image

    if len(results[0].boxes) == 0: # if cant recognize card
        print("No card detected!")
        return None

    # set 4 corner location
    x1, y1, x2, y2 = map(int, results[0].boxes[0].xyxy[0])
    cropped_card = image[y1:y2, x1:x2]

    r = rotate_idcard_model.predict(source=cropped_card, show=False, conf=0.35)

    result = r[0]
    boxes = result.boxes.xyxy.cpu().numpy().astype(int)
    confidences = result.boxes.conf.cpu().numpy()
    class_ids = result.boxes.cls.cpu().numpy().astype(int)
    class_names = result.names

    image = cropped_card

    cv2.imshow(image)

    flag_x1, flag_y1, flag_x2, flag_y2 = boxes[0]
    flag_center_x = (flag_x1 + flag_x2) // 2
    flag_center_y = (flag_y1 + flag_y2) // 2

    flag_class_id = class_ids[0]
    flag_class_name = class_names[flag_class_id]

    logo_x1, logo_y1, logo_x2, logo_y2 = boxes[1]
    logo_center_x = (logo_x1 + logo_x2) // 2
    logo_center_y = (logo_y1 + logo_y2) // 2

    logo_class_id = class_ids[1]
    logo_class_name = class_names[logo_class_id]

    cv2.rectangle(image, (flag_x1, flag_y1), (flag_x2, flag_y2), (0, 255, 0), 2)
    cv2.putText(image, flag_class_name, (flag_x1, flag_y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
    cv2.circle(image, (flag_center_x, flag_center_y), 5, (0, 255, 0), -1)

    cv2.rectangle(image, (logo_x1, logo_y1), (logo_x2, logo_y2), (255, 0, 0), 2)
    cv2.putText(image, logo_class_name, (logo_x1, logo_y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)
    cv2.circle(image, (logo_center_x, logo_center_y), 5, (255, 0, 0), -1)

    slope = (logo_center_y - flag_center_y) / (logo_center_x - flag_center_x)
    angle = np.arctan(slope)

    # if logo_center_x < flag_center_x:
    #     rotation_angle = np.degrees(angle) + 180

    # else:
    #     rotation_angle = np.degrees(angle)

    rotation_angle = np.degrees(angle)

    center = (image.shape[1] // 2, image.shape[0] // 2)

    rotation_matrix = cv2.getRotationMatrix2D(center, rotation_angle, 1.0)
    rotated_image = cv2.warpAffine(image, rotation_matrix, (image.shape[1], image.shape[0]))

    r_new = rotate_idcard_model.predict(source=rotated_image, show=False, conf=0.2)
    result = r_new[0]

```



```

boxes = result.bboxes.xyxy.cpu().numpy().astype(int)

flag_x1n, flag_y1n, flag_x2n, flag_y2n = boxes[0]
logo_x1n, logo_y1n, logo_x2n, logo_y2n = boxes[1]

new_flag_center = (flag_x1n + flag_x2n) // 2
new_logo_center = (logo_x1n + logo_x2n) // 2

if new_flag_center > new_logo_center:
    rotation_angle = np.degrees(angle) + 180
else:
    rotation_angle = np.degrees(angle)

rotation_matrix = cv2.getRotationMatrix2D(center, rotation_angle, 1.0)
rotated_image = cv2.warpAffine(image, rotation_matrix, (image.shape[1], image.shape[0]))

cv2.imshow(rotated_image)
return(rotated_image)

```

بعد از انجام تغییرات مورد نیاز برای سیستم چرخش و همینطور آماده کردن مدل segmentation برای تشخیص و جدا کردن کارت بقیه مراحل برای اعمال پرسپکتیو ساده بود که در زیر مشاهده میشود مراحل شامل به دست آوردن نسبت اضلاع، تشخیص مقدار مورد نیاز برای اعمال تغییر، ساخت ماتریس تبدیل و همینطور اعمال یک تغییر برای بهینه کردن گوشه ها و سپس اعمال تغییر اصلی روی عکس که البته این روش مقداری باعث کاهش کیفیت تصویر میشود ولی به اندازه زیادی تاثیر گزار نخواهد بود

```

aspect_ratio = w / h
# set output size based on aspect ratio
if aspect_ratio > 1: # wider than tall
    width = 1180
    height = int(width / aspect_ratio)
else: # taller than wide
    height = 750
    width = int(height * aspect_ratio)

# define ideal rectangular coordinates for the output
ideal_coords = np.array([[0, 0], [width, 0], [width, height], [0, height]], dtype="float32")

# compute perspective transformation matrix
matrix = cv2.getPerspectiveTransform(np.array(ordered_corners, dtype="float32"), ideal_coords)

# apply sharpening filter to enhance edges
kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
sharpened = cv2.filter2D(image, -1, kernel)

# warp the sharpened image to the corrected perspective
corrected_image = cv2.warpPerspective(sharpened, matrix, (width, height), flags=cv2.INTER_LANCZOS4)

```

جدا کردن نواحی دارای اطلاعات:

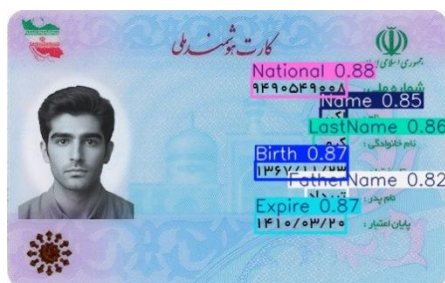
ما تصمیم گرفتیم برای اینکه بتوانیم خروجی بهتری از مدل OCR بگیریم، نواحی حاوی اطلاعات کلیدی (مثل نام، نام خانوادگی، کد ملی و غیره) را به صورت جداگانه به مدل بدهیم. این روش باعث می شود که مدل OCR به جای پردازش کل تصویر کارت، فقط روی بخش های مهم و مشخص تمرکز کند. این کار

دقت استخراج متن را افزایش می‌دهد و از پردازش غیرضروری قسمت‌های بدون اطلاعات جلوگیری می‌کند. برای حل این مشکل، تصمیم گرفتیم یک مدل دیگر آموزش دهیم که بتواند نواحی حاوی اطلاعات را به صورت مستقل شناسایی کند. مرحله آماده کردن دیتاست هم که همانطور که قبلاً اشاره شد انجام گرفت و تعداد بسیار خوبی دیتا آماده شد.

```
!yolo task=detect mode=train model=yolov8s.pt data={dataset.location}/data.yaml epochs=30 imgsz=640
```

در این مرحله هم تونستیم دقت را بالای 90 درصد نگه داریم بعد از آموزش، مدل توانست نواحی حاوی اطلاعات را با دقت بالای 90 درصد شناسایی کند. حالا که نواحی متنی به صورت جداگانه شناسایی می‌شوند، می‌توانیم هر کدام را مستقل به مدل OCR بدهیم و دقت استخراج متن را به میزان قابل‌توجهی بهبود دهیم.

یک مثال از کارکرد مدل (شکل 7 تشخیص شکل 5 است)



7 Figure

مدل OCR برای تشخیص متن

بی‌تردید، مهم‌ترین بخش این پروژه، مرحله تشخیص متن بود، زیرا عملکرد مطلوب این مرحله، شرط موفقیت تمامی مراحل پیشین به شمار می‌رفت. به همین دلیل، تغییرات متعددی در این بخش اعمال شد و بخش قابل‌توجهی از اصلاحات صورت‌گرفته در مراحل قبلی نیز تحت تأثیر نیاز به بهبود عملکرد مدل OCR انجام گرفت.

پس از بررسی و جست‌وجو در منابع مرتبط، با دو کتابخانه برجسته و شناخته‌شده مواجه شدیم: EasyOCR و PyTesseract. با انجام چندین آزمایش اولیه و ارزیابی خروجی‌ها، مشخص شد که EasyOCR در تشخیص متون پارسی دقت بهتری نسبت به PyTesseract ارائه می‌دهد. بنابراین، این کتابخانه به عنوان ابزار اولیه ما برای تشخیص متن انتخاب شد. EasyOCR امکانات ارزشمندی در اختیار ما قرار داد؛ از جمله، قابلیت `rotate_info` که به صورت خودکار چرخش تصویر را تشخیص داده و مناسب‌ترین خروجی را ارائه می‌کرد. همچنین، ارائه `Confidence Score` برای هر تشخیص، امکان ارزیابی کیفیت خروجی‌ها را به ما می‌داد و دید مناسبی نسبت به عملکرد مدل فراهم می‌کرد.

با وجود این مزایا، پس از آزمایش‌های بیشتر و استفاده از ورودی‌های پیچیده‌تر، متوجه شدیم که عملکرد EasyOCR در زبان پارسی به سطح انتظارات ما نمی‌رسد. به‌طور خاص، این کتابخانه در تشخیص عدد "5" ناتوان بود و در تمامی موارد، به‌جای آن عدد "0" را در خروجی نمایش می‌داد. به دلیل این محدودیت‌ها و دقت ناکافی، تصمیم گرفتیم استفاده از این مدل را متوقف کنیم.

در ادامه، با جست‌وجوهای گسترده‌تر و تمرکز ویژه بر زبان پارسی، با مدل هزار (Hezar) آشنا شدیم. این مدل متن‌باز، که توسط توسعه‌دهندگان ایرانی طراحی و به‌طور اختصاصی برای زبان پارسی توسعه یافته است، از دقت فوق‌العاده بالایی در تشخیص نوری کاراکترها (OCR) برخوردار بود و به نظر می‌رسید بهترین گزینه برای پروژه ما باشد. دقت این مدل در مقایسه با EasyOCR به مراتب بالاتر بود و نتایج آن به‌طور چشمگیری قابل‌توجه و شگفت‌انگیز بودند.

با این حال، مدل هزار نیز با چالش‌هایی همراه بود. نخست، این مدل فاقد دو قابلیت کلیدی ارائه‌شده توسط EasyOCR بود: یک متد داخلی برای تشخیص و اعمال خودکار چرخش تصویر، و ارائه Confidence Score دقیق. امتیاز اطمینان (Confidence) ارائه‌شده توسط مدل هزار از دقت کافی برخوردار نبود و اغلب اعدادی بالای 80 درصد را نشان می‌داد، در حالی که خروجی‌ها در برخی موارد به وضوح دقتی کمتر از 5 درصد داشتند. این موضوع، قابلیت تحلیل و اعتماد به این داده‌ها را برای ما از بین می‌برد. چالش دوم این بود که مدل هزار به ورودی‌هایی کاملاً تراز نیاز داشت و زوایای بیشتر از 5 تا 10 درجه در تصویر ورودی، به خروجی‌هایی کاملاً نامرتب منجر می‌شد. مشکل سوم نیز عملکرد ضعیف مدل در پردازش تصاویر با کیفیت پایین، دارای سایه، نویز یا سایر اختلالات بود که کارایی آن را تحت تأثیر قرار می‌داد.

برای رفع مشکل زاویه تصاویر ورودی، راهکارها و تغییراتی که در بخش‌های پیشین توضیح داده شد، اعمال گردید. همچنین، برای بهبود عملکرد مدل در برابر نویز و کیفیت پایین تصاویر، تصمیم گرفتیم مدل هزار را Fine-Tune کنیم. با وارد کردن مجموعه‌ای از تصاویر با کیفیت پایین‌تر به عنوان داده‌های آموزشی، تلاش کردیم تا توانایی مدل را در پردازش این نوع ورودی‌ها ارتقا دهیم. این اقدامات به ما کمک کرد تا محدودیت‌های مدل را تا حدی برطرف کرده و دقت کلی سیستم را بهبود ببخشیم.

مراحل Fine-Tune مدل هزار:

آماده‌سازی دیتاست

برای آماده‌سازی مجموعه داده مورد نیاز برای تنظیم دقیق مدل، از رویکردی ترکیبی استفاده شد که شامل تولید داده‌های مصنوعی و تقویت آن‌ها بود. در مرحله اول، بخشی از تصویر پشت کارت ملی به عنوان پس‌زمینه انتخاب شد تا شرایط واقعی اسناد هویتی شبیه‌سازی شود. سپس، برای تولید متن روی این پس‌زمینه، از یک فایل CSV حاوی اسامی متنوع استفاده شد که این اسامی از طریق ChatGPT جمع‌آوری شده بودند. علاوه بر این، تاریخ‌ها و کدهای ملی به صورت تصادفی ایجاد شدند و بر روی پس‌زمینه قرار گرفتند تا تنوع داده‌ها افزایش یابد.

Figure 9 کد استفاده نده برای تولید تصاویر

```
def generate(text, bg):
    image_size = (200, 70)

    bg = Image.open(f'/content/drive/MyDrive/ImageGenerate/{bg}.jpg')
    bg = bg.resize(image_size, Image.Resampling.LANCZOS)
    draw = ImageDraw.Draw(bg)

    font = ImageFont.truetype('/content/drive/MyDrive/ImageGenerate/iransansdn.ttf', 40)

    bbox = draw.textbbox((0, 0), text, font=font)
    text_width = bbox[2] - bbox[0]
    text_height = bbox[3] - bbox[1]

    position = ((image_size[0] - text_width) // 2, ((image_size[1] - text_height) // 2) - 17)

    draw.text(position, text, font=font, fill=(0, 0, 0))

    t = text.replace('/', '')

    output_path = f'/content/drive/MyDrive/ImageGenerate/images/{t}.jpg'
    bg.save(output_path)

    bg_bright = ImageEnhance.Brightness(bg).enhance(1.5)
    output_path_bright = f'/content/drive/MyDrive/ImageGenerate/images/{t}b.jpg'
    bg_bright.save(output_path_bright)

    bg_blur = bg.filter(ImageFilter.GaussianBlur(radius=1.4))
    output_path_blur = f'/content/drive/MyDrive/ImageGenerate/images/{t}.jpg'
    bg_blur.save(output_path_blur)
```

به منظور بهبود تعمیم‌پذیری مدل و شبیه‌سازی شرایط مختلف تصویربرداری، از (Data Augmentation) استفاده شد. این تکنیک‌ها شامل تغییر پرسپکتیو، چرخش، تنظیم روشنایی و افزودن نویز بود. این فرآیند نه تنها حجم مجموعه داده را افزایش داد، بلکه مدل را قادر ساخت تا با تصاویر دارای تنوع و پیچیدگی بیشتر سازگار شود. در نهایت، مجموعه داده تولیدشده به سه بخش آموزش (70%)، اعتبارسنجی (15%)، و آزمون (15%) تقسیم شد تا مراحل آموزش و ارزیابی به طور جامع انجام شود.

مراحل انجام کد

در این بخش، کد ارائه‌شده به صورت گام‌به‌گام توضیح داده می‌شود تا درک کاملی از فرآیند تنظیم دقیق مدل CRNN ارائه شود:

فایل کد:

<https://github.com/cactus1386/NationalCard-ImageProcessing/blob/main/code/trainOCR.ipynb>

تعریف بلوک CNN: کلاس CNNBlock یک واحد پایه برای شبکه کانولوشنی (CNN) تعریف می‌کند که شامل یک لایه کانولوشنی (Conv2d)، نرمال‌سازی دسته‌ای (BatchNorm2d)، و تابع فعال‌سازی ReLU است. این بلوک‌ها برای استخراج ویژگی‌های بصری از تصاویر ورودی استفاده می‌شوند.

تعریف مدل CRNN: کلاس CRNN معماری اصلی مدل را پیاده‌سازی می‌کند که ترکیبی از (CNN)، (RNN)، و یک لایه خطی برای طبقه‌بندی است. این مدل ابتدا ویژگی‌ها را از تصاویر استخراج می‌کند، سپس آن‌ها را به یک توالی تبدیل کرده و با استفاده از لایه‌های LSTM دوجهته، وابستگی‌های متوالی را مدل‌سازی می‌کند. در نهایت، خروجی به تعداد کلاس‌های کاراکترها (تعداد کاراکترهای قابل تشخیص جدا می‌شود).

پیش‌پردازش تصاویر: تابع preprocess_image تصاویر ورودی را به فرمت مناسب برای مدل

تبدیل می‌کند. این شامل تبدیل تصویر به مقیاس خاکستری، تغییر اندازه به 32×384 ، اعمال آینه‌سازی (در صورت نیاز)، نرمال‌سازی، و مقیاس‌بندی بر اساس تنظیمات فایل پیکربندی است. ایجاد مجموعه داده: کلاس `NewOCRDataset` مجموعه داده را تعریف می‌کند. این کلاس مسیر تصاویر را بارگذاری کرده، برچسب‌ها را از نام فایل‌ها استخراج می‌کند و آن‌ها را به شناسه‌های عددی بر اساس نگاشت کاراکترها در فایل پیکربندی تبدیل می‌کند. تابع `format_label` برای تبدیل فرمت خاص برچسب‌ها (مانند تاریخ یا کد ملی) استفاده می‌شود.

آماده‌سازی داده‌ها برای آموزش: تابع `collate_fn` داده‌ها را به صورت دسته‌ای (Batch) سازمان‌دهی می‌کند تا برای ورودی به مدل مناسب باشند. مجموعه داده به سه بخش آموزش، اعتبار‌سنجی، و آزمون تقسیم شده و با استفاده از `DataLoader` برای پردازش دسته‌ای آماده می‌شود.

آموزش و تنظیم دقیق مدل: تابع `train_model` فرآیند آموزش را مدیریت می‌کند. این تابع از معیار `CTC Loss` برای محاسبه خطا استفاده می‌کند، که برای مسائل تشخیص متوالی مانند `OCR` مناسب است. بهینه‌ساز `Adam` با نرخ یادگیری `0.0001` به کار گرفته شده و مکانیزم توقف زودهنگام (`Early Stopping`) پیاده‌سازی شده است تا از بیش‌برازش جلوگیری شود. بهترین مدل در طول آموزش ذخیره می‌شود.

ارزیابی روی داده‌های آزمون: پس از آموزش، مدل روی مجموعه داده آزمون ارزیابی می‌شود. خروجی مدل با استفاده از تابع Log Softmax پردازش شده و خطای CTC محاسبه می‌شود تا عملکرد نهایی مدل سنجیده شود.

نکات تکمیلی

چالش‌ها: یکی از مشکلات مشاهده‌شده در کد، خطای نوع داده در زمان ارزیابی بود که به دلیل عدم انتقال داده‌های آزمون به دستگاه GPU رخ داد. این مشکل با انتقال مستقیم داده‌ها به دستگاه مناسب قابل حل است.

بهبودها: می‌توان با افزایش تنوع داده‌ها یا استفاده از تکنیک‌های پیشرفته‌تر تقویت داده، دقت مدل را بیشتر بهبود داد.

مقایسه خروجی برنامه روی دیتاست فاز 2 بعد از اعمال تغییرات روی برنامه فاز 2

[illegible][illegible]

Figure 10 برنامه ای که در فاز 2 ارائه شد

Figure 11 برنامه ارائه شده برای آخرین مرحله

به طور کلی، نتایج آخرین آزمایش های انجام شده در این پروژه رضایت بخش بوده و نشان دهنده پیشرفت قابل توجهی نسبت به نسخه های پیشین است. بهره گیری از دو مدل YOLO که هر یک به دقتی بیش از 90 درصد دست یافتند، استفاده از یک مدل پردازش تصویر که تعادل مطلوبی بین دقت و سرعت برقرار کرد، و همچنین تنظیم دقیق (Fine-Tuning) مدل هزار، همگی در کنار یکدیگر به افزایش چشمگیر دقت و سرعت سیستم منجر شدند و خروجی هایی با کیفیت و رضایت بخش به همراه داشتند.



با این حال، پروژه همچنان ظرفیت بهبود و توسعه بیشتری دارد. در حال حاضر، سیستم در مواجهه با تصاویر بسیار چالش برانگیز، و قادر به ارائه نتایج مطلوب نیست.

Figure 12 یکی از تصاویر سخت که مدل همچنان در تشخیص آن مشکل دارد

علاوه بر این، امکان اعمال تغییراتی برای افزایش کاربردپذیری پروژه در دنیای واقعی نیز وجود دارد. به عنوان مثال، می توان قابلیت هایی نظیر تحلیل کد ملی برای اعتبارسنجی و تشخیص صحت آن، اتصال به پایگاه های داده موجود برای انجام فرآیندهای احراز هویت، و افزودن ابزارهای صحت سنجی مانند تشخیص واقعی بودن تصویر کارت ملی در برابر تصاویر تولید شده دیجیتالی یا شناسایی کارت های تقلبی را به سیستم

اضافه کرد. این بهبودها می‌توانند پروژه را به یک ابزار جامع و بومی تشخیص هویت تبدیل کنند که توانایی رقابت با راحل‌های پیشرفته خارجی، مانند Sumsb، را داشته باشد.

استفاده از هوش مصنوعی:

ابزار ها زبانی در تمام مسیر پروژه در کنار ما بودند و استفاده های زیادی از ان ها انجام گرفت

بیشتر تحقیق ها و جست و جو های مورد نیاز ها قابلیت ها Deepthink , Deepserach , مدل های ChatGPT و Grok انجام گرفت

همچنین در موارد بسیاری به دلیل برخوردن به ارور های عجیب و غیرمنتظره مجبور به استفاده و کمک گرفتن از این ابزار ها شدیم.

و مهمترین بخشی که این ابزار ها مورد استفاده قرار گرفتند بخش انجام پرسپکتیو و البته تغییر زاویه بود.

مخصوصا بخش اعمال تغییرات پرسپکتیو که بیشتر مراحل که شامل محاسبات و استفاده از ابزار های بیشتر بود از هوش مصنوعی به مقدار خیلی زیادی کمک گرفته شد

منابع مرتبط و ارجاعات:

لینک کامل پروژه در گیت‌هاب که تمام مراحل که طی شده در آن قابل مشاهده است:

<https://github.com/cactus1386/NationalCard-ImageProcessing>

گیت‌هاب علی نجف پور:

<https://github.com/AliNajafpour>

گیت‌هاب رادین الماسی:

<https://github.com/cactus1386>

منابع ،مقالات و ابزارهای استفاده شده:

<https://github.com/serengil/deepface>

<https://medium.com/@appanamukesh77/comprehensive-insights-onfacial-analysis-with-insightface-library-796d80464f45>

<https://medium.com/@maritaganta/how-to-properly-rotate-an-image-with-opencv-475e44a252f6>

<https://pillow.readthedocs.io/en/stable/reference/ImageDraw.html>

<https://www.sussex.ac.uk/ei/internal/forstudents/engineeringdesign/studyguides/techreportwriting>

[/https://docs.ultralytics.com](https://docs.ultralytics.com)

<https://medium.com/@saranshrajput/face-detection-using-mtcnn-f3948e5d1acb>

<https://github.com/dilippuri/PAN-Card-OCR>

https://www.youtube.com/watch?v=6Hj9vakUsUY&list=PL4sqggbpSjuJjZz_YyNx5e2jPrR9kun2l8&index=5

https://www.youtube.com/watch?v=qCUCmA3thl0&list=PL4sqggbpSjuJjZz_YyNx5e2jPrR9kun2l8&index=14

https://www.youtube.com/watch?v=3fwJ3Mk9Nz0&list=PL4sqggbpSjuJjZz_YyNx5e2jPrR9kun2l8&index=12

[/https://github.com](https://github.com)

[/https://colab.research.google.com](https://colab.research.google.com)

[/https://roboflow.com](https://roboflow.com)